

1. Participant Information & Topic Name

- Team Name [to fill]
- Team Lead Name [to fill]
- Team Lead Contact No [to fill]

Selected Track

- ☐ Quantum Machine Learning (QML)
- **Post-Quantum Cryptography (PQC)**
- ☐ Web3 & Blockchain
- ☐ Generative AI & Machine Learning

Q-SHIELD

Continuous device trust with post-quantum-signed evidence

Authentication answers **who are you** once.
Q-SHIELD keeps asking **can you still be trusted.**



ML-KEM-768 and ML-DSA-65 (FIPS 203, 204)

2. Problem Definition

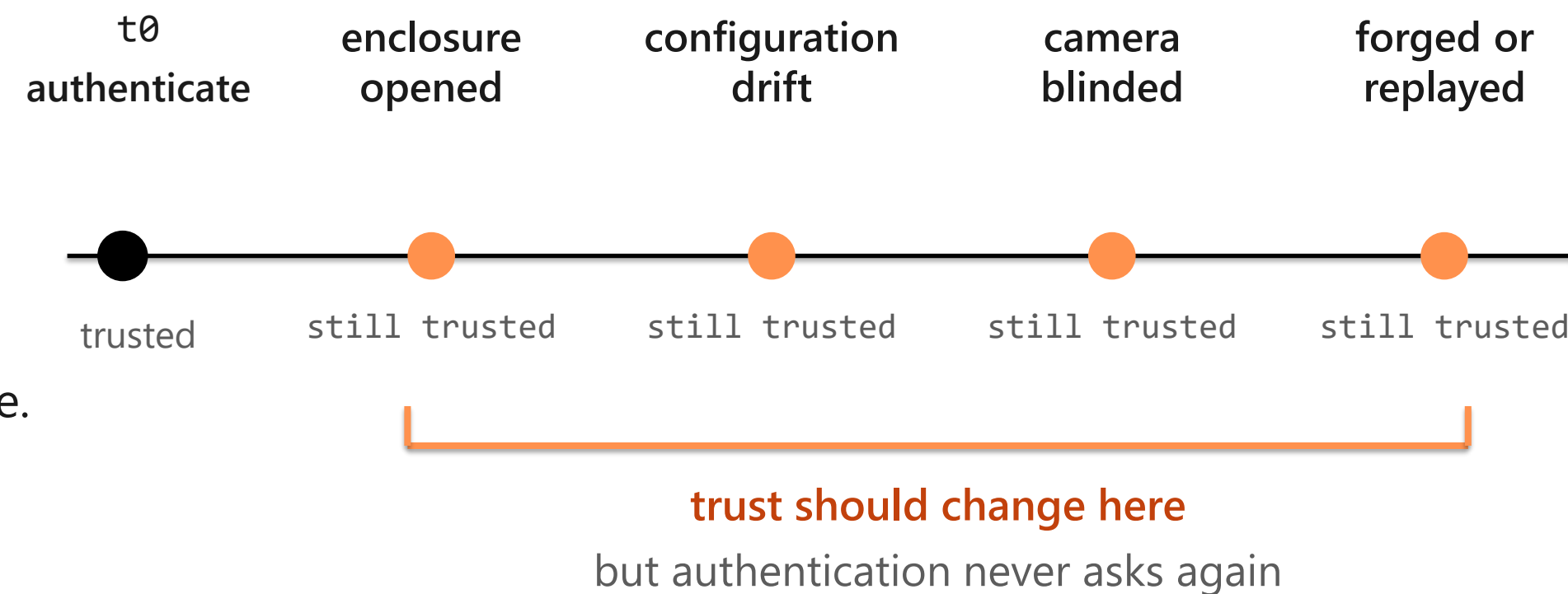
Problem description: authenticated is not the same as trustworthy

A device passes authentication. Then:

- its enclosure is opened
- its configuration drifts
- its camera is covered or turned
- its messages are forged or replayed

One-time authentication still says **trusted**.

Evidence recorded today must also stay unforgeable when quantum computers arrive.



2. Problem Definition

Why authentication alone is not enough

Threat model: what each attack looks like, and what answers it

Threat	Authentication alone	What Q-SHIELD adds
Forged message	rejected, then forgotten	security event + bounded pressure (at most 25 points)
Replayed message or observation	rejected, then forgotten	pressure; 3 replays in 10 min hold the score at 65
Enclosure opened (tamper switch)	nothing to see	physical factor CRITICAL; score held at 55
Camera covered, frozen or turned	nothing to see	signed camera-health report; visual factor HIGH
Tamper + visual evidence within 60 s	nothing to see	confirmed incident: score 30, quarantine
Quantum adversary recording traffic	classical signatures forgeable later	ML-DSA-65 signs the evidence; ML-KEM-768 keys the session

2. Problem Definition

Functional requirements and constraints

What the system must do, and the limits it was built under

Functional requirements

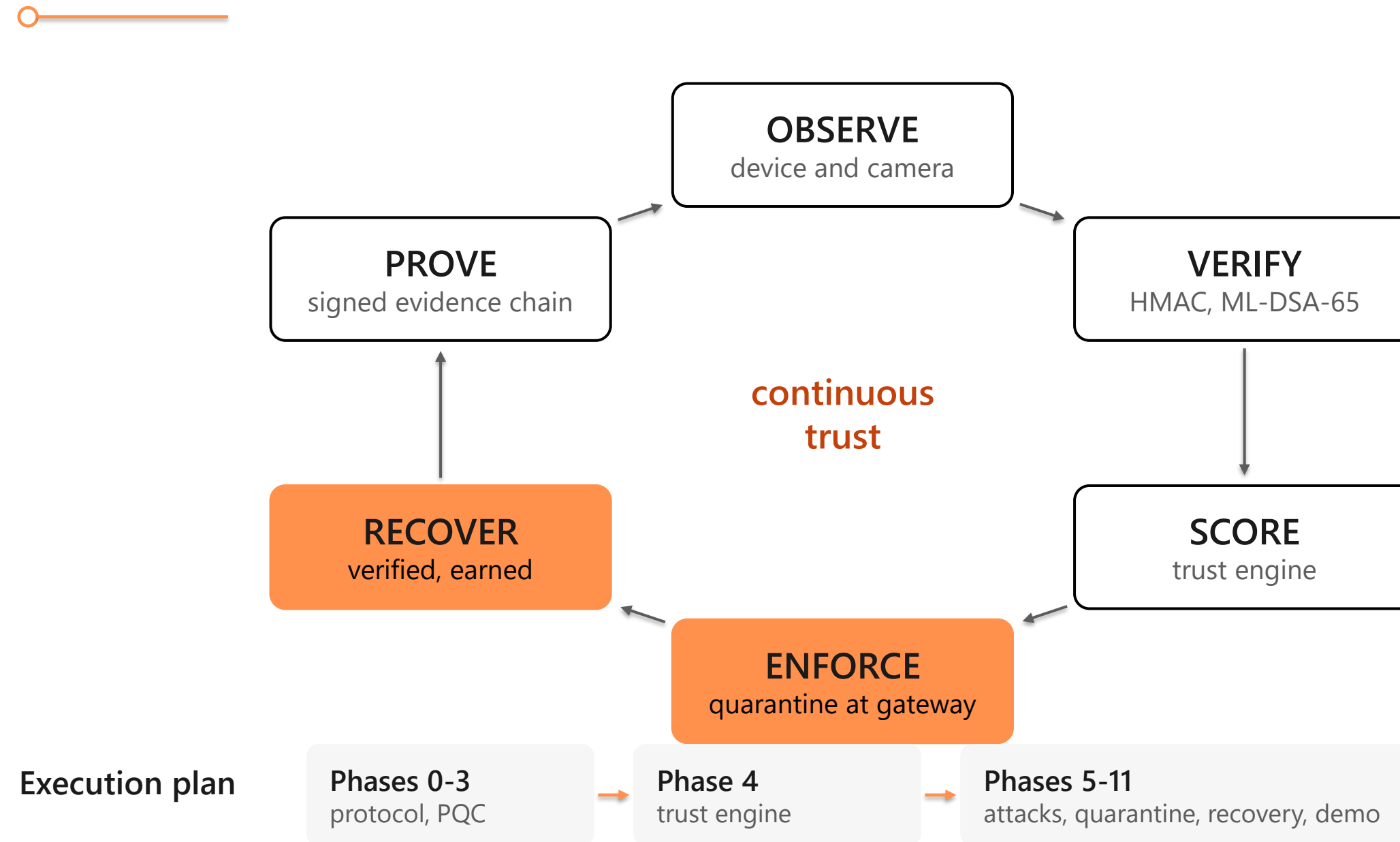
- **Continuous scoring**
six evidence factors, every change explained
- **Cross-modal correlation**
two independent signals within 60 s confirm an incident
- **Gateway enforcement**
quarantine blocks the device's normal channel
- **Earned recovery**
health checks, then trust rebuilt from fresh evidence
- **Tamper-evident record**
SHA-256 linked, ML-DSA-65 signed evidence chain
- **Post-quantum evidence path**
ML-KEM-768 session, ML-DSA-65 signatures

Constraints

- **IoT-class device**
device path is HMAC-SHA256, not post-quantum
- **No hardware attestation**
device state is self-reported evidence
- **Laptop gateway**
FastAPI + SQLite, single process
- **No physical device yet**
telemetry comes from a labelled software agent
- **Uncalibrated parameters**
weights and thresholds are documented design choices

3. Proposed Solution & Execution Plan

Solution overview: a closed loop around every device



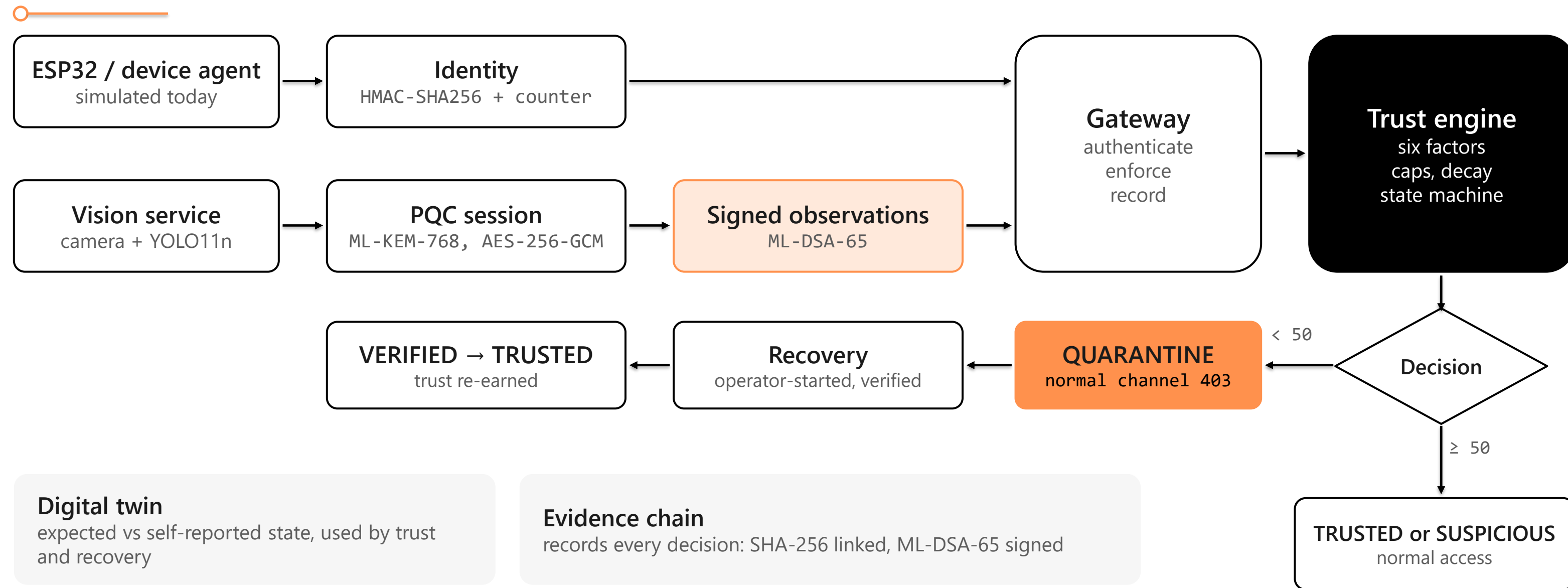
Working principle

- Every message is authenticated before it counts
- Every decision is scored and explained
- Every block is enforced at the gateway
- Every change is signed into the chain
- Trust comes back only with fresh evidence

3. Proposed Solution & Execution Plan

Working principle: system architecture

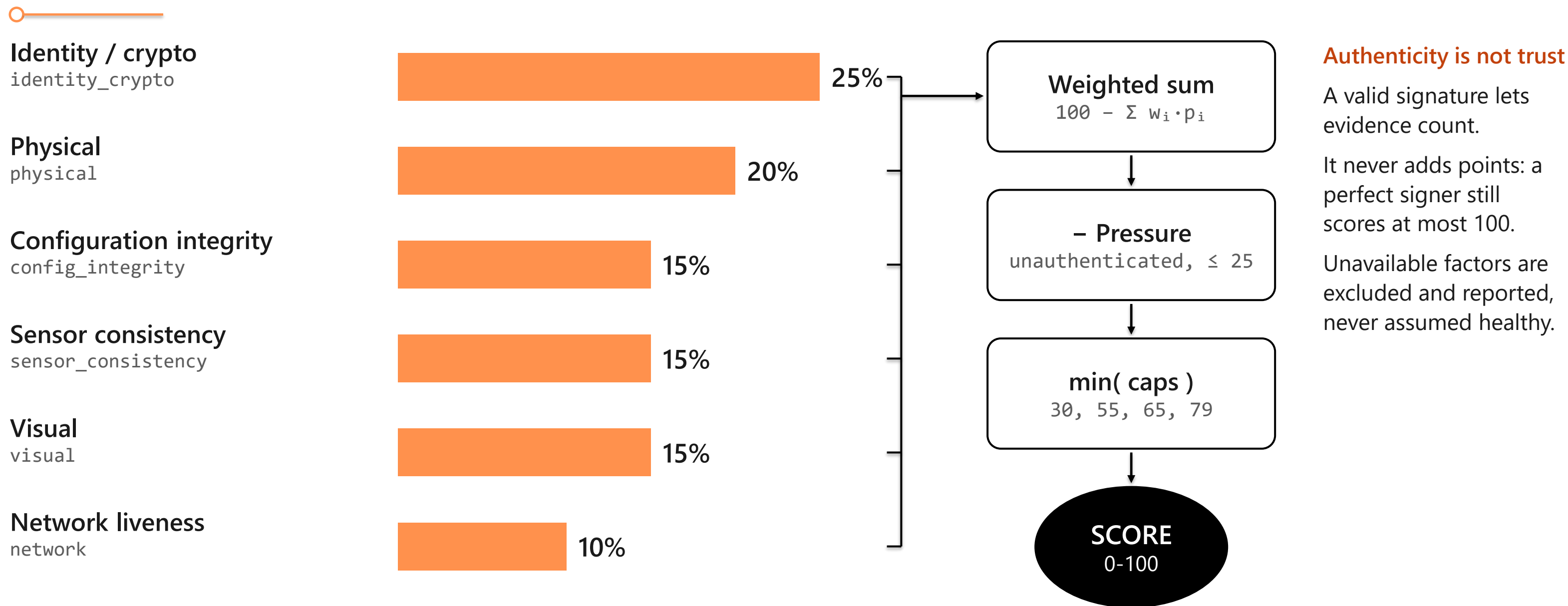
Two clients authenticate differently and never interchangeably



3. Proposed Solution & Execution Plan

Core functionality: the six-factor trust model

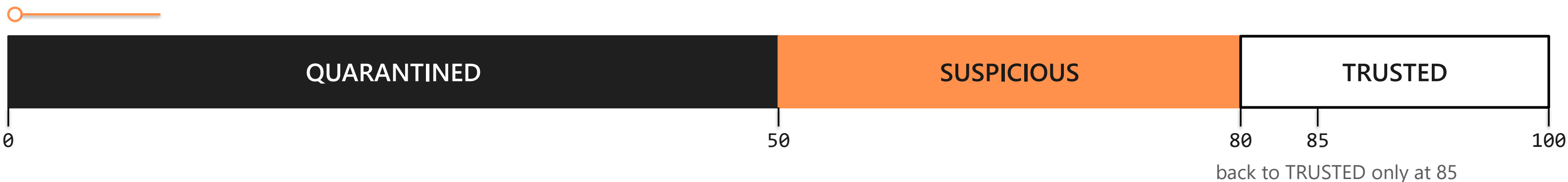
Each factor is scored from its own evidence



3. Proposed Solution & Execution Plan

Continuous trust scoring

From a number to a state, with caps that hold while a condition lasts



```
raw      = 100 - Σ wi · pi
final    = min(raw - pressure, caps)
pressure ≤ 25  (can never quarantine alone)
score    = round(final)
```

Caps (ceiling while active)

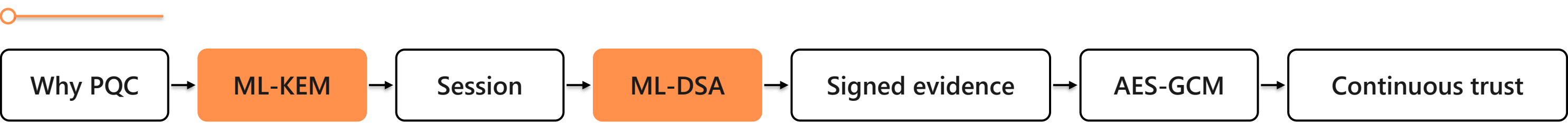
Confirmed incident	30
Physical tamper	55
Correlated incident	55
Repeated replay	65
Integrity mismatch	65
Stale device	79

Recovery credit: $\Delta c = \text{clamp}(t - \max(t_{\text{prev}}, t_{\text{violation}}), 0, 45 \text{ s})$

Only clean, authenticated device evidence earns time. An interval containing a violation earns nothing; silence never heals a device.

4. Technical Design

Technologies and algorithms: why post-quantum, and where each primitive works



Use	Algorithm	Standard	Where
Session key	ML-KEM-768	FIPS 203	vision service ↔ gateway
Observation signatures	ML-DSA-65	FIPS 204	vision service identity
Evidence signatures	ML-DSA-65	FIPS 204	gateway evidence key
Key schedule	HKDF-SHA256	RFC 5869	session keys, per direction
Session data	AES-256-GCM	SP 800-38D	observations in transit
Chain links	SHA-256	FIPS 180-4	evidence chain
Device messages	HMAC-SHA256	RFC 2104	ESP32 path, not PQC

70

NIST ACVP vectors agree

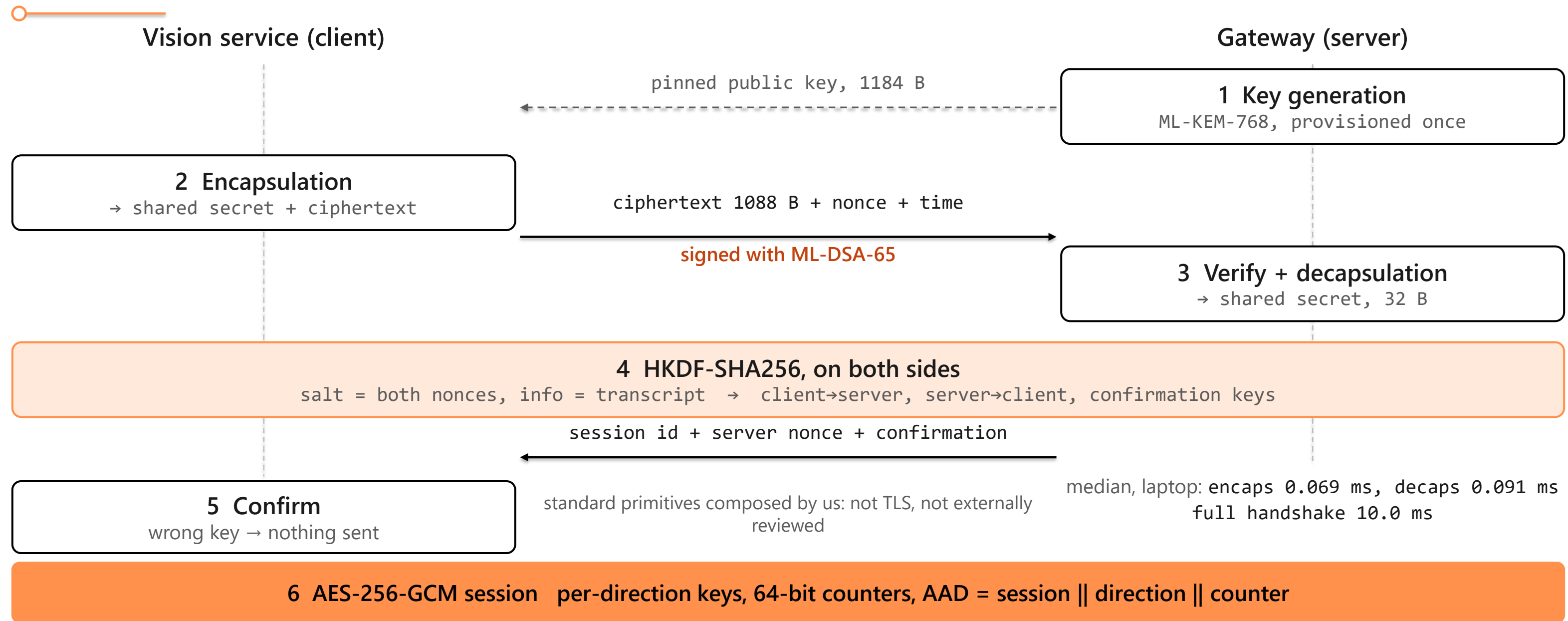
- 35 ML-KEM-768 decapsulation
- 20 ML-KEM-768 key checks
- 15 ML-DSA-65 verifications

Algorithm-level evidence, not a certification.

4. Technical Design

ML-KEM-768 handshake flow

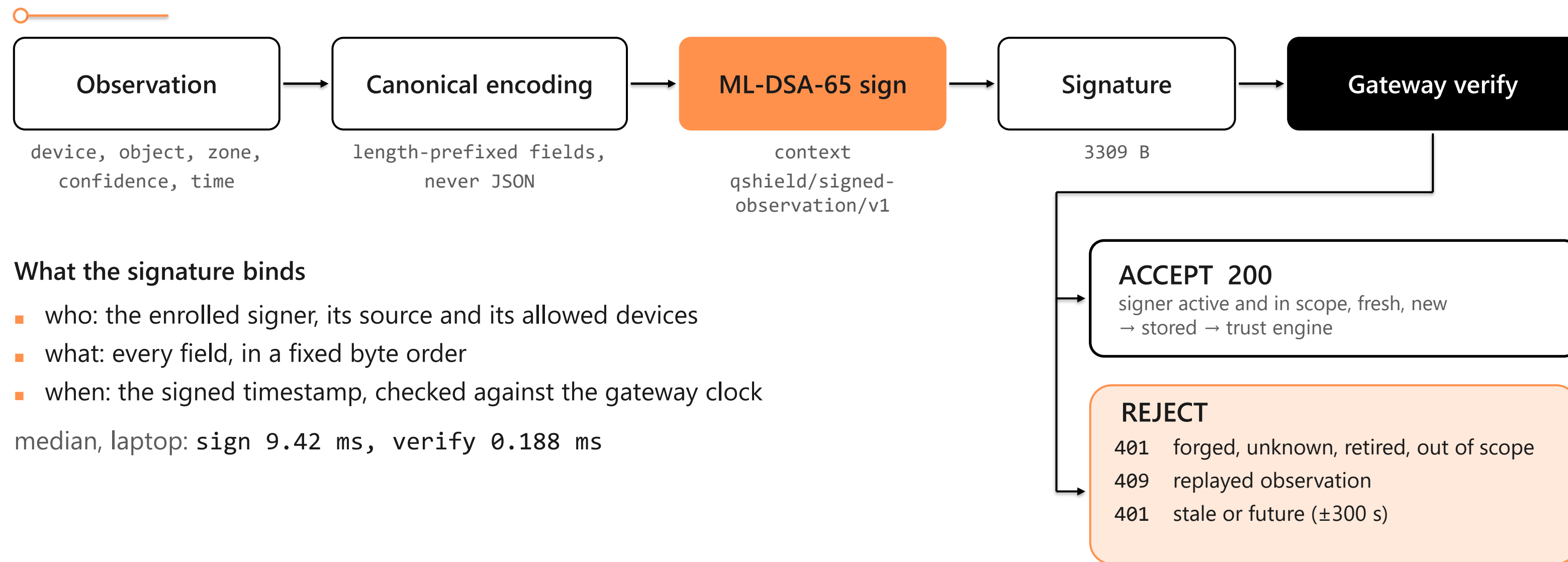
Vision service and gateway agree on session keys; the init is ML-DSA-65 signed



4. Technical Design

ML-DSA-65 signed observation flow

From a camera observation to an accepted or rejected record



What the signature binds

- who: the enrolled signer, its source and its allowed devices
- what: every field, in a fixed byte order
- when: the signed timestamp, checked against the gateway clock

median, laptop: sign 9.42 ms, verify 0.188 ms

4. Technical Design

Session security and abuse resistance

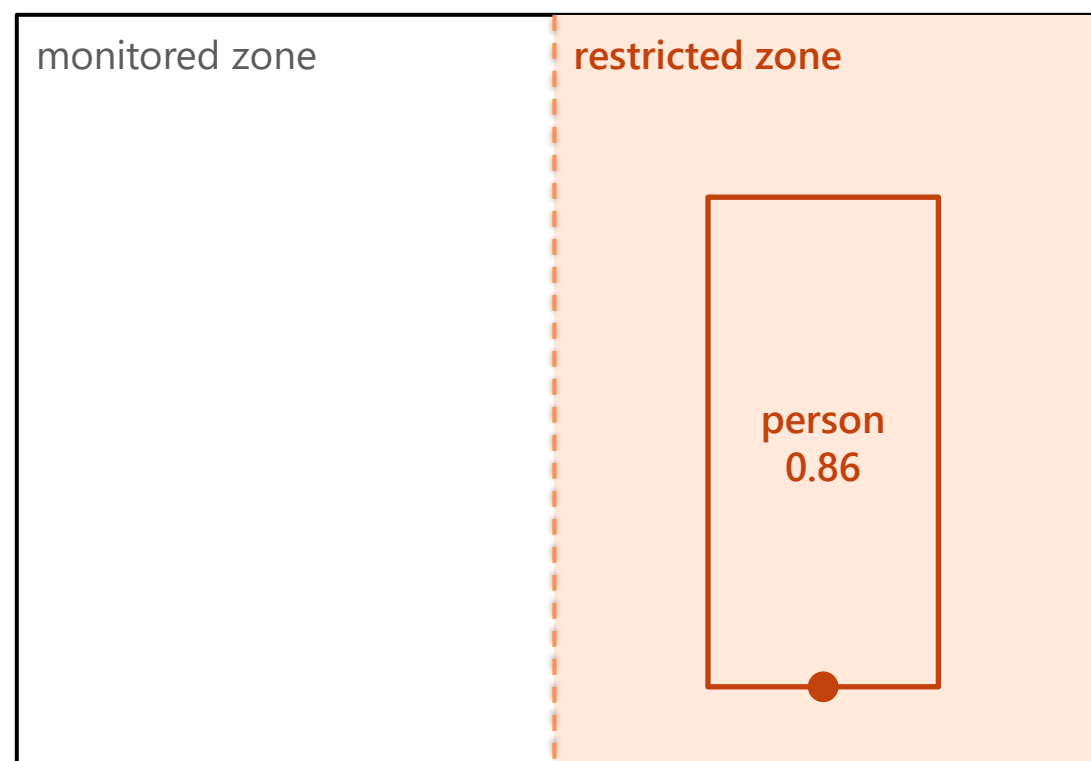
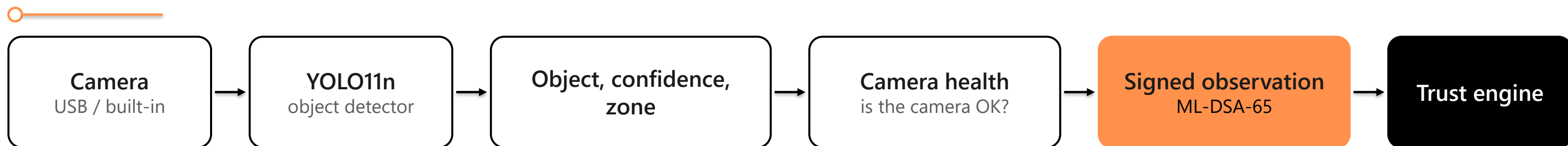
What an attacker can do to the transport, and what stops it

Attack	Control	Tested result
Replayed handshake	one-use nonce, reserved atomically	concurrent replay gets one session
Replayed or reordered message	strict 64-bit counters, AES-GCM tag	replay and reorder rejected
Gateway lost the session	401 session_expired → one re-handshake	observation still delivered
Persistent refusals	drop that observation, keep the session	30 refusals: 1 session (was 30)
Session-table exhaustion	gateway cap 64; client 20 handshakes/h, backoff 1-60 s	budget holds under a loop
Rejection flood	3 samples per sender per 10 s, rest coalesced with counts	300 forged messages: 8 rows

4. Technical Design

AI vision security

The detector reports what it sees; only the trust engine decides what it means



anchor: bottom-centre of the box

Rule match = restricted class in a restricted zone
a policy match, not an anomaly-detection verdict

The webcam has no key

the vision service holds the ML-DSA-65 identity, scoped to its device

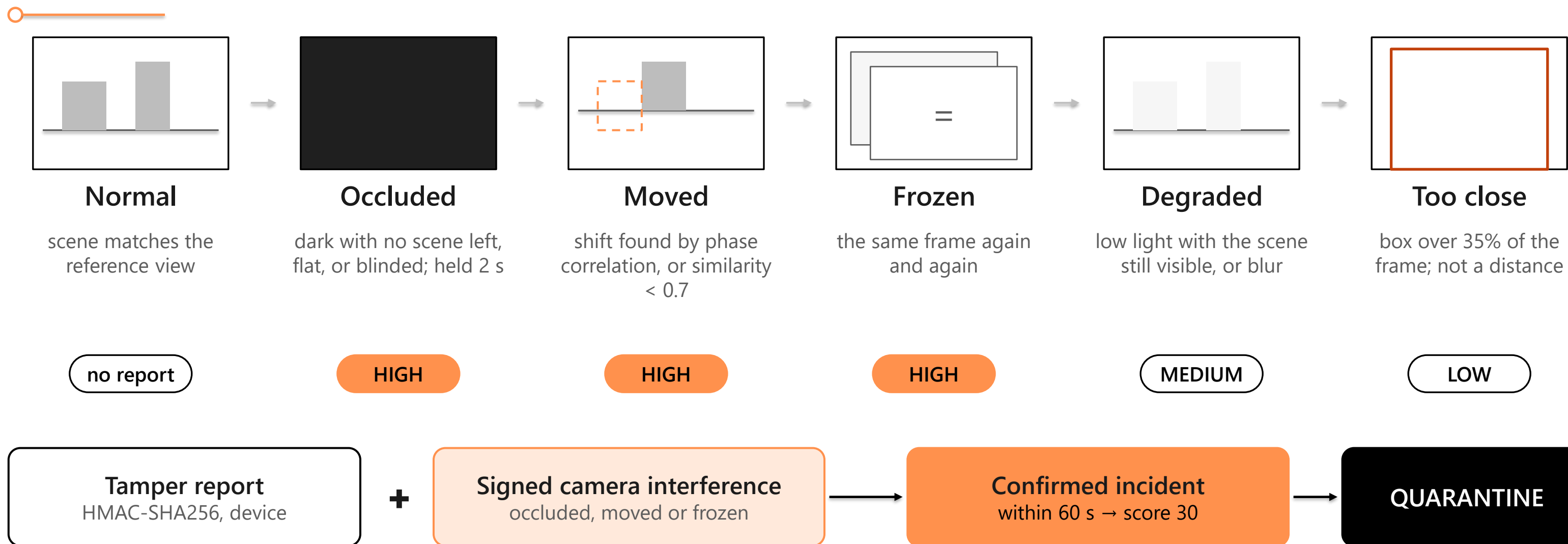
Live on a USB webcam, CPU only

about 5 frames/s at the configured rate; detection accuracy not measured

4. Technical Design

Camera tamper and anomaly model

The camera is part of the security boundary, not just a detector



Camera alone: a penalty, never quarantine. Camera + rule match from the same signer: one modality, so a stolen vision key cannot confirm an incident by itself.

4. Technical Design

Digital twin

Expected state, set by an operator, against what the device reports (authenticated)

Field	Expected (operator)	Observed (device report)	Verdict
Firmware	agent-0.1	agent-0.1	MATCH
Configuration	cfg-good-1	cfg-tampered-0000	MISMATCH
Capabilities	tamper, temperature, vibration	tamper, temperature, vibration	MATCH
Temperature	-20 to 60 °C	27.4 °C	MATCH
Tamper state	closed	open	MISMATCH

How it is used

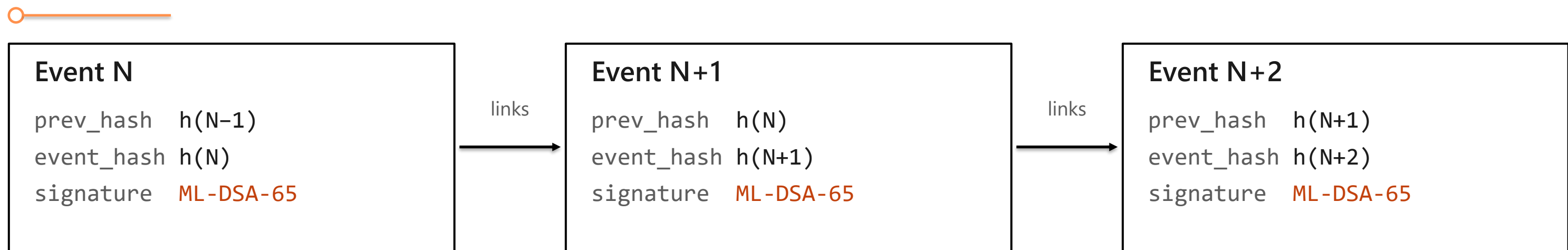
- overrides the global trust expectations per device
- recovery checks judge each report on its own content
- the known-good state is frozen during recovery

A match is evidence, not attestation.

4. Technical Design

Evidence chain and forensics

Every decision is linked by SHA-256 and signed with ML-DSA-65



Signature on every entry

context qshield/evidence/v1, gateway evidence key

Verification

recompute every SHA-256 hash, check every signature
→ an edited, reordered or removed entry is found

Live demo result

26 entries, chain **VERIFIED**

Not anchored externally: cutting entries off the end needs an outside copy of the head to detect, and whoever holds both the database and the evidence key could rewrite history.

4. Technical Design

Quarantine enforcement

Applied at the gateway, after authentication, on three separate channels

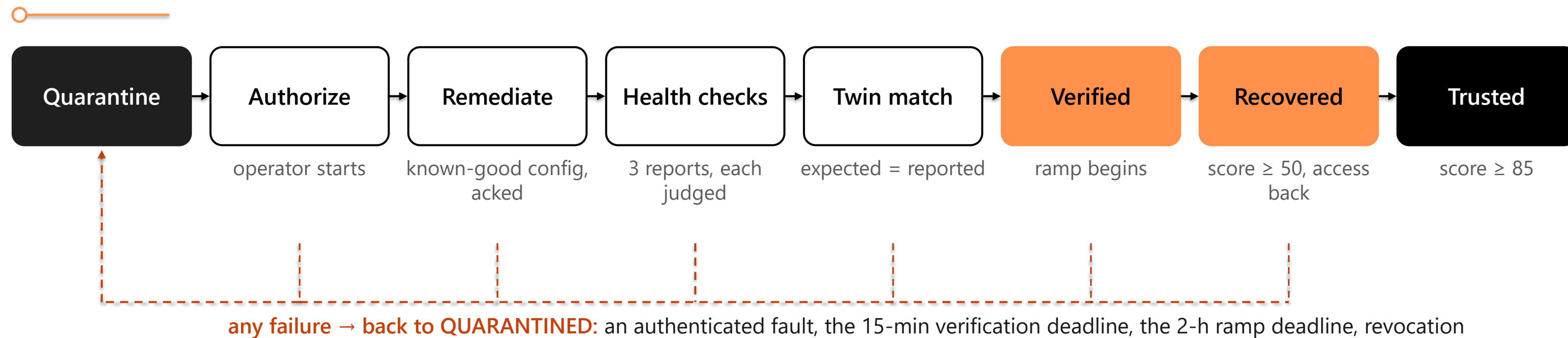
	Normal channel register, heartbeat, telemetry	Recovery channel recovery/report	Control operators only
TRUSTED, SUSPICIOUS, RECOVERED	allowed	refused: not open	named operator
QUARANTINED, RECOVERING, VERIFIED	403 blocked	allowed	named operator
Forged or replayed message	401 before enforcement	401 before enforcement	401

Forged traffic can neither trigger quarantine nor bypass it: it is rejected by authentication before the policy runs.
Blocked messages are not trust evidence; recovery-channel reports are.

4. Technical Design

Self-healing recovery

Trust is earned back with fresh evidence, never granted



Rules

- each health check judges that report alone: a stale twin field cannot pass it
- only the recovery channel stays open while quarantined
- software remediation only: it cannot repair hardware

Live demo (announced time-lapse)

health checks 3 of 3 → **VERIFIED** at score 30

RECOVERED score 80 after 30 min of credited evidence

TRUSTED score 85 after 42 min

4. Technical Design

Technology stack

Grouped by what each part does in the system

Cryptography

pqcrypto ML-KEM-768, ML-DSA-65
cryptography AES-256-GCM, HKDF-SHA256
hashlib, hmac SHA-256 chain, HMAC device auth

AI / vision

OpenCV camera capture
YOLO11n (ultralitics) object detection
NumPy camera-health statistics

Backend

FastAPI + uvicorn gateway API
Pydantic strict schemas
Trust engine pure Python, deterministic

Frontend

Vanilla ES modules dashboard, no build step
Presentation mode one-screen live view

Database

SQLite events, observations, twin, evidence chain, trust state

Deployment

One laptop gateway optional TLS, named operators

Hardware (planned): ESP32 skeleton (not compiled); BME280, MPU6050, reed switch

5. Feasibility & Innovation

Technical feasibility: what is real today, and what is simulated

Real and tested

- ML-KEM-768 and ML-DSA-65 (pqcrypto)
- AES-256-GCM, HKDF-SHA256
- SHA-256 linked evidence chain
- YOLO11n vision on a real webcam
- trust engine and recovery engine
- digital twin and quarantine enforcement
- dashboard and browser camera preview

Simulated

- device telemetry (software agent, labelled)
- attacks (attack_simulation, labelled)
- ESP32 deployment
- physical sensors and tamper switch

Hardware is an adapter, not a redesign

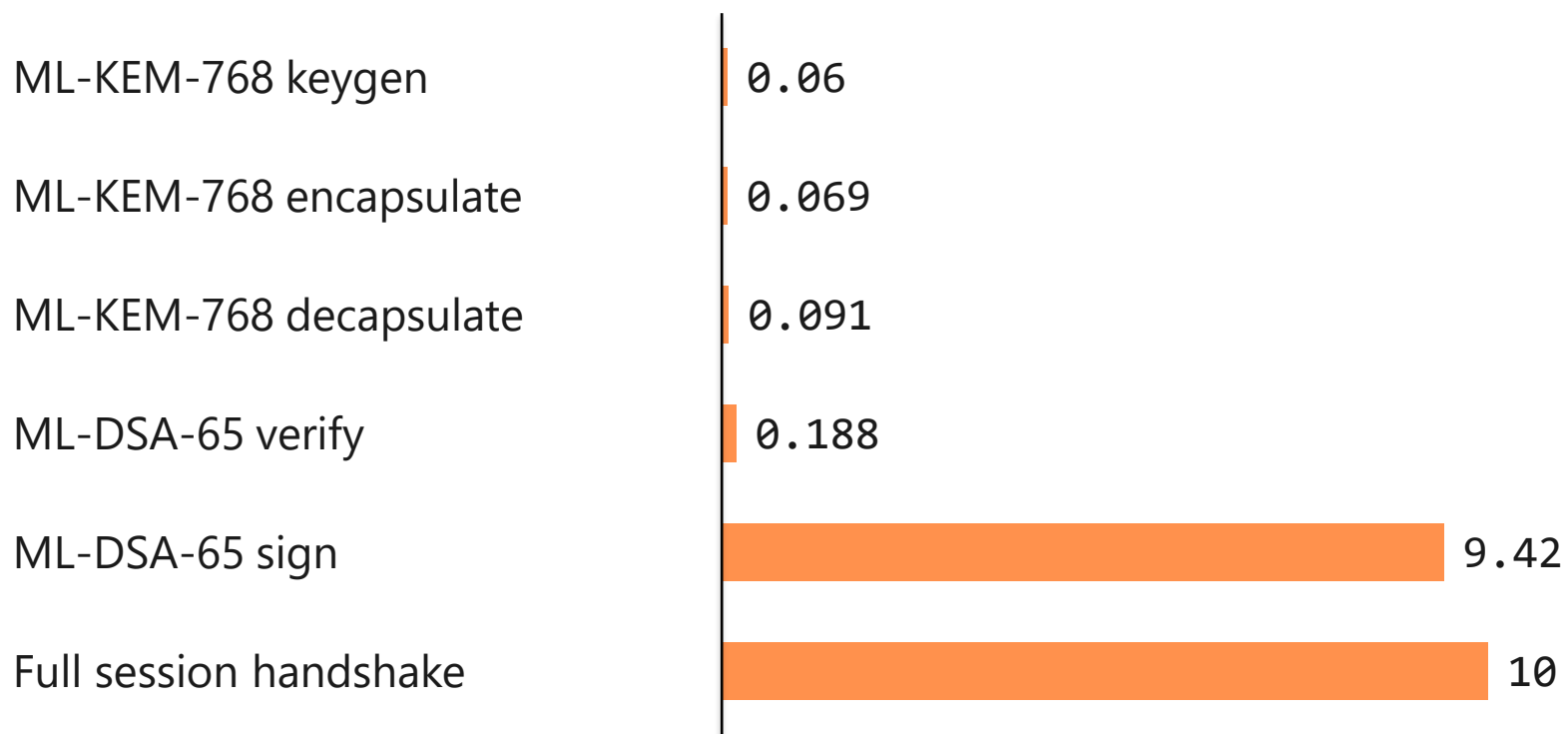
- same envelope, same gateway, same trust engine
- 7 byte-exact protocol test vectors (tests/vectors)
- ESP32-CAM stream: a configuration change (not yet tried)

5. Feasibility & Innovation

Performance and testing

Scalability and performance: measured numbers only

Post-quantum operations, median ms (laptop CPU)



Key exchange is about 100 times cheaper than signing. i7-13700HX, Python bindings; nothing measured on an ESP32.

785

Python tests passing

40

JavaScript tests passing

70

NIST ACVP vectors agree

53 μ s

trust-engine update (mean)

Tests by area

trust 235, security 149, AI 135, full stack 111, PQC 74, integration 47, twin 18, evidence 15

Scale, honestly: one gateway on SQLite, one process; multi-gateway is not built.

5. Feasibility & Innovation

Innovation / USP

What Q-SHIELD does that one-time authentication cannot

- **Continuous, explainable trust**
every score change lists the reasons that sum to it
- **Authenticity is not trust**
signatures decide what counts; they never add points
- **Camera as a security boundary**
covered, frozen or turned is signed evidence
- **Bounded grieving**
forged traffic is capped at 25 points: it can never quarantine
- **Earned recovery, signed evidence**
post-quantum signatures on every decision

The novelty is the loop

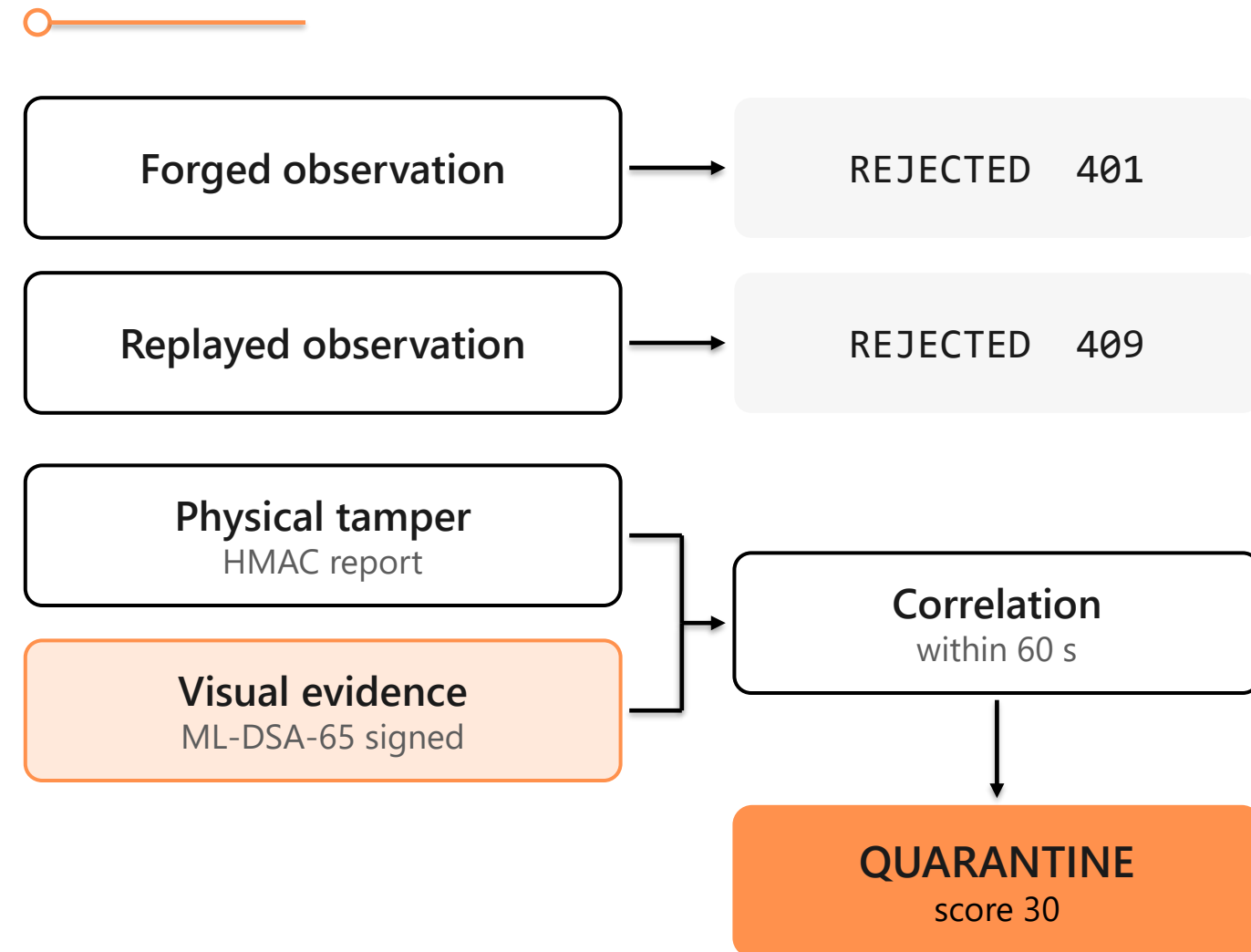
observe → verify → score → enforce →
recover → prove

Each mechanism is standard. Their integration around one device's continuous trust is the contribution.

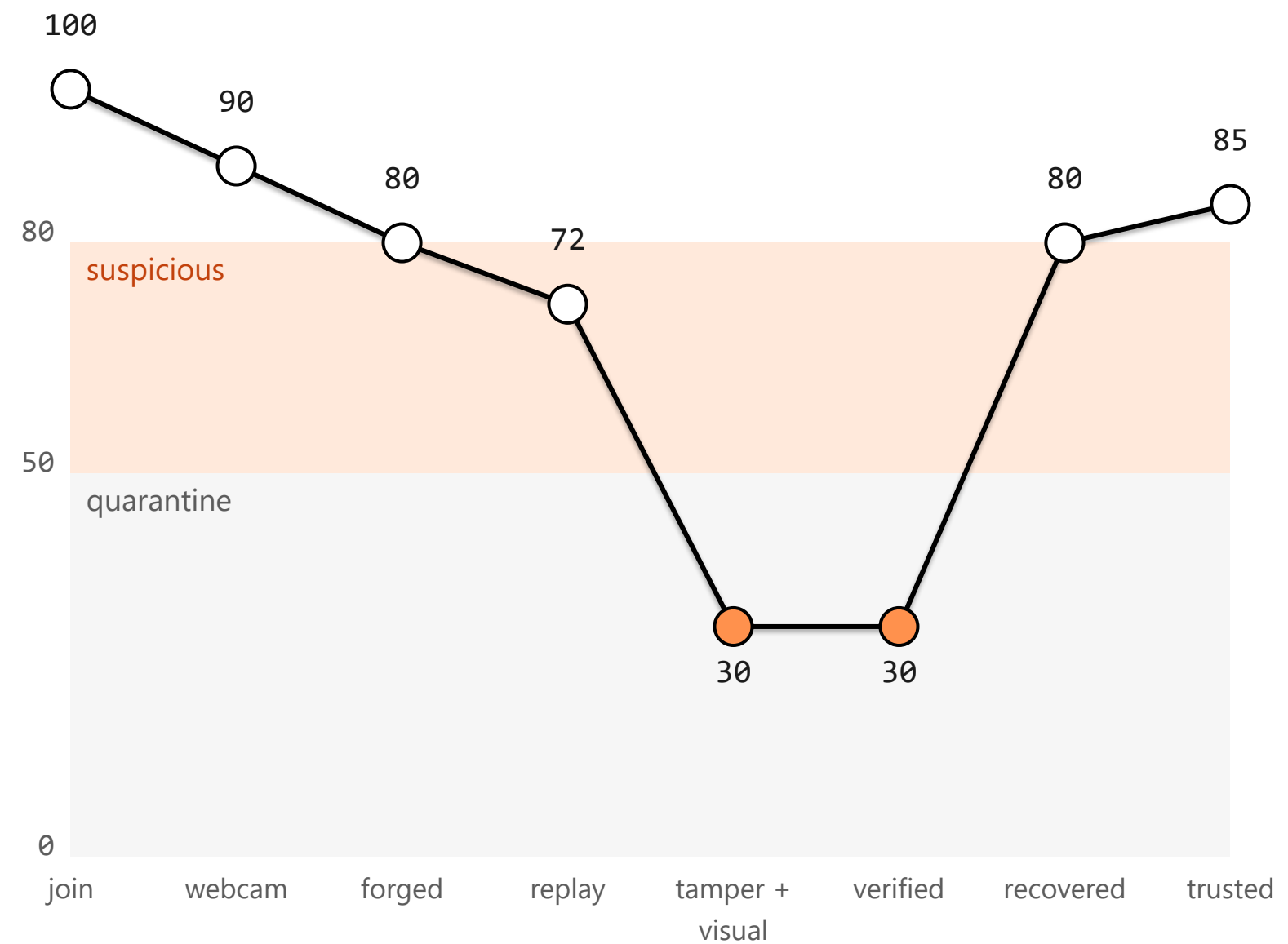
A prior-art survey has not been done; we claim an integration, not a new algorithm.

6. Expected Outcome

Expected results / prototype: the live demo, end to end



Real HTTP against the real gateway. Attacks are simulated and labelled; the webcam detections are real.



Live run on the USB webcam, 2026-10-02; recovery shown in announced time-lapse

6. Expected Outcome

Real-world impact and future scope

What is still missing, what comes next, and why it matters

Honest boundaries

- device telemetry simulated; ESP32 never compiled
- device path HMAC-SHA256, not post-quantum
- no hardware attestation
- camera thresholds unvalidated; proximity is not a distance
- evidence chain not anchored externally
- single gateway; trust parameters uncalibrated

Next, with hardware

- 1 compile and flash; match the 7 test vectors
- 2 wire the reed switch, then BME280 and MPU6050
- 3 stage camera tests: cover, turn, unplug, approach
- 4 measure, then calibrate thresholds
- 5 anchor the evidence head externally

Impact

- labs, server rooms and critical sites get devices that must keep earning trust
- incidents arrive with a signed, explainable trail
- evidence stays unforgeable into the quantum era

Authenticate once. Verify continuously. Recover only with evidence.